String Loops: A Problem-Solving Investigation

Ron

1 Introduction

In this report I investigate the following problem.

Problem. In a bowl there are n strings of equal length. I repeatedly choose two free ends uniformly at random and tie them together. When all free ends have been tied, some closed string loops are formed. Investigate the number of loops obtained.

This report is written as a problem-solving investigation rather than as a research essay. My aim is not to search for external results, but to describe how I approached the problem myself: what I first understood, which small cases I tried, what patterns I thought I saw, where I got stuck, and what eventually led to a complete description of the process.

The problem looked simple at first, but I quickly realised that there are two rather different ways to think about it. One way is to look at the final pairing of the $2n$ endpoints. Another way is to track how the partially tied strings evolve during the process. My first attempts mostly used the first point of view, but the breakthrough came from switching to the second.

Throughout the report I write

$$L_n = \text{the number of loops formed from } n \text{ strings.}$$

So the central goal is to understand the probability distribution of L_n, and in particular to find formulas for its expectation and variance and to explain why the distribution has the shape it does.

The report is organised as follows. I begin by identifying what is known and what I want to find. I then specialise to small values of n, since this is the most natural way to get a first sense of the problem. After that I explain where direct counting becomes awkward and why I became stuck. I then introduce a more useful description in terms of open chains, prove the main theorem, derive consequences, and finally check the results numerically using Python simulations.

2 Entry: what I know, what I want, and what I introduce

I found it helpful to organise the start of the problem under the headings what I know, what I want, and what I introduce.

What I know

Initially there are n separate strings. Each string has two free ends, so there are $2n$ free ends in total. At each stage, two of the currently free ends are chosen uniformly at random and tied together. The process continues until no free ends remain. At that point, the final object is a disjoint union of closed loops.

A first immediate observation is that the final number of loops must lie between 1 and n. It can never be 0, because eventually all chains must close. It can never exceed n, because there were only n original strings to begin with.

What I want

I want to understand the random variable L_n. More specifically, I want to answer the following questions:

- What is the exact distribution of L_n?
- Can I derive a recurrence for the probabilities?
- What are $\mathbb{E}[L_n]$ and $\mathrm{Var}(L_n)$?
- What kind of probability distribution is this?
- Can numerical simulation support the theoretical results?

What I introduce

I introduce the notation

$$P_n(\ell) = \mathbb{P}(L_n = \ell), \qquad \ell = 1, 2, \ldots, n.$$

I also introduce the strategy of *specialising* to small cases. Since the process is random and combinatorial, I found it natural to begin with $n = 1, 2, 3, 4, 5$. This let me build a table of exact values and use it to guide my later conjectures.

3 Specialising to small cases

My first serious step was to compute the exact outcomes for the smallest values of n. This was useful for two reasons. First, it helped me avoid making vague guesses about the general behaviour. Second, it gave me a table of concrete data that I could later check against any theoretical formula.

3.1 Case $n = 1$

This case is immediate. There is only one string, and its two free ends must be tied together. Therefore

$$P_1(1) = 1.$$

So $L_1 = 1$ with certainty.

3.2 Case $n = 2$

Now there are 4 endpoints in total, and there are 3 possible pairings. Of these:

- one pairing ties each string to itself, giving 2 loops;
- the other two pairings link the strings into one larger loop.

Hence

$$P_2(1) = \frac{2}{3}, \qquad P_2(2) = \frac{1}{3}.$$

At this point I already noticed an interesting fact: even when there are only two strings, the most likely outcome is not two small loops but one larger loop.

3.3 Case $n = 3$

For $n = 3$, direct case analysis gives

$$P_3(1) = \frac{8}{15}, \qquad P_3(2) = \frac{2}{5}, \qquad P_3(3) = \frac{1}{15}.$$

This was my first clear sign that the probability of getting many loops becomes small quite quickly. In particular, the probability of getting the maximum number 3 of loops is already only 1/15.

3.4 Cases $n = 4$ and $n = 5$

Continuing in the same way gave

$$P_4(1) = \frac{16}{35}, \qquad P_4(2) = \frac{44}{105}, \qquad P_4(3) = \frac{4}{35}, \qquad P_4(4) = \frac{1}{105},$$

and

$$P_5(1) = \frac{128}{315}, \qquad P_5(2) = \frac{80}{189}, \qquad P_5(3) = \frac{4}{27}, \qquad P_5(4) = \frac{4}{189}, \qquad P_5(5) = \frac{1}{945}.$$

These values are summarised in Table 1.

n	$P_n(1)$	$P_n(2)$	$P_n(3)$	$P_n(4)$	$P_n(5)$
1	1	–	–	–	–
2	$\frac{2}{3}$	$\frac{1}{3}$	–	–	–
3	$\frac{8}{15}$	$\frac{2}{5}$	$\frac{1}{15}$	–	–
4	$\frac{16}{35}$	$\frac{44}{105}$	$\frac{4}{35}$	$\frac{1}{105}$	–
5	$\frac{128}{315}$	$\frac{80}{189}$	$\frac{4}{27}$	$\frac{4}{189}$	$\frac{1}{945}$

Table 1: Exact distributions for L_n for $n = 1, \ldots, 5$.

3.5 What I observed from the table

At this stage I did not yet have a proof of anything general, but several patterns stood out.

- The support of L_n appears to be exactly $\{1, 2, \ldots, n\}$.
- The probability of many loops is small.
- The most likely values stay relatively small even as n increases.
- The expected number of loops appears to grow slowly.

To make the last point more precise, I computed the first few expectations directly from the table:

$$\mathbb{E}[L_1] = 1, \qquad \mathbb{E}[L_2] = \frac{4}{3}, \qquad \mathbb{E}[L_3] = \frac{23}{15},$$

$$\mathbb{E}[L_4] = \frac{176}{105}, \qquad \mathbb{E}[L_5] = \frac{5632}{3150} \approx 1.787.$$

These values increase, but very slowly. This made me suspect that the mean might grow logarithmically rather than linearly.

4 Conjectures, failed ideas, and where I got stuck

After building the table above, I tried to guess a general formula directly from it. This was useful, even though my first ideas turned out to be wrong, because it showed me what kind of reasoning was not sufficient.

4.1 Direct counting becomes awkward

My first approach was to count all complete pairings of the $2n$ endpoints. The number of pairings is

$$(2n-1)!! = (2n-1)(2n-3)\cdots 3 \cdot 1.$$

So for small n this is manageable, but it grows very quickly. I could still work through $n = 4$ and $n = 5$, but it was becoming increasingly clear that this was not a good route to a general understanding.

The problem was not only that the numbers got large. More importantly, direct counting did not explain *why* the probabilities had the pattern seen in the table.

4.2 An incorrect binomial-style intuition

For a while, I thought the number of loops might behave like a binomial random variable. After all, one might imagine that at each stage there is some chance of "creating a loop" and some chance of "not creating a loop". This sounds binomial in spirit.

However, when I checked this more carefully, the idea failed. A binomial distribution would require the same success probability at each stage, but nothing in the process suggests that the probability of creating a new loop should stay constant throughout. In fact, the table already hinted that the probabilities must depend strongly on how many open parts remain.

So this was my first real stuck point: the final table clearly had structure, but it was not standard binomial structure.

4.3 The important question

At that point I realised that I had been focusing too much on the final outcome and not enough on the evolving process. The right question was no longer

"What does the final pairing look like?"

but rather

"What happens at each intermediate stage?"

This shift of viewpoint turned out to be the crucial breakthrough.

5 A better description: open chains

The key idea is to describe the partially tied strings as *open chains*.

Initially, each string is itself an open chain with two free ends, so there are n open chains. Whenever I tie two free ends together, exactly one of the following happens:

- two different open chains are joined into one longer open chain;
- the two free ends of the same open chain are tied together, and that chain becomes a closed loop.

In either case, the number of open chains decreases by exactly 1.

This means that the process must pass through the states

$$n,\ n-1,\ n-2,\ \ldots,\ 2,\ 1,$$

where the number indicates how many open chains remain.

Once I wrote the problem in this way, the most important quantity became clear:

When there are k open chains, what is the probability that the next tie creates a loop?

That question is much easier than trying to describe the full pairing all at once.

6 Main theorem

I now state the main result of the investigation.

Theorem 1. *For $n \geq 1$,*

$$L_n \stackrel{d}{=} 1 + \sum_{k=2}^{n} X_k,$$

where the random variables $X_2, X_3, \ldots, X_n$ are independent and

$$X_k \sim \text{Bernoulli}\left(\frac{1}{2k-1}\right).$$

Proof. For each $k = 1, 2, \ldots, n$, consider the moment when there are exactly k open chains remaining. Define

$$X_k = \begin{cases} 1, & \text{if the next tie creates a new loop,} \\ 0, & \text{otherwise.} \end{cases}$$

When $k = 1$, the last remaining open chain must close into a loop, so $X_1 = 1$.

Now suppose $k \geq 2$. At this stage there are $2k$ free ends. Fix one free end. It can be tied to any of the other $2k-1$ free ends, all with equal probability. Exactly one of these $2k-1$ endpoints is the other free end of the same open chain. If that particular endpoint is chosen, a loop is created immediately. Any other choice joins two different chains together without creating a loop.

Therefore

$$\mathbb{P}(X_k = 1) = \frac{1}{2k-1}, \qquad \mathbb{P}(X_k = 0) = \frac{2k-2}{2k-1}.$$

The total number of loops formed is exactly the number of stages at which a loop is created. Hence

$$L_n = \sum_{k=1}^{n} X_k = 1 + \sum_{k=2}^{n} X_k.$$

Finally, to justify independence, note that for each stage k, once there are k open chains remaining, the probability that the next tie creates a loop is always

$$\frac{1}{2k-1},$$

regardless of how that state was reached. In other words, for any fixed outcomes of the later-stage variables $X_{k+1}, X_{k+2}, \ldots, X_n$, we still have

$$\mathbb{P}(X_k = 1 \mid X_{k+1}, X_{k+2}, \ldots, X_n) = \frac{1}{2k-1}.$$

So the conditional distribution of X_k does not depend on the earlier history. Repeating this argument for each k shows that the joint distribution factors into the product of the Bernoulli probabilities, and hence $X_2, \ldots, X_n$ are independent. $\square$

6.1 Why this theorem is important

This theorem completely explains the probabilistic structure of the problem.

It shows that the number of loops is not something mysterious arising only at the very end. Instead, it is built stage by stage. At each stage k, either a new loop is formed or it is not, and the probability of success at that stage is simply

$$\frac{1}{2k-1}.$$

This also explains why my earlier binomial guess was close in spirit but ultimately wrong. The distribution really does come from Bernoulli-type events, but the success probability changes with the stage.

7 Immediate consequences

Corollary 1. *The expectation and variance of L_n are*

$$\mathbb{E}[L_n] = \sum_{k=1}^{n} \frac{1}{2k-1},$$

and

$$\mathrm{Var}(L_n) = \sum_{k=2}^{n} \frac{1}{2k-1}\left(1 - \frac{1}{2k-1}\right).$$

Proof. These formulas follow immediately from the theorem, because L_n is a sum of independent Bernoulli random variables. □

7.1 Interpretation as a distribution

Since L_n is the sum of independent Bernoulli variables with different parameters, it is not binomial. More precisely, it is a shifted *Poisson-binomial* random variable:

$$L_n = 1 + \sum_{k=2}^{n} X_k, \qquad X_k \sim \mathrm{Bernoulli}\left(\frac{1}{2k-1}\right).$$

I found this satisfying because it gave a clean name and structure to the distribution I had first encountered only as a table of fractions.

7.2 Checking the expectation against small cases

The expectation formula also matches the small values found earlier:

$$\mathbb{E}[L_1] = 1,$$

$$\mathbb{E}[L_2] = 1 + \frac{1}{3} = \frac{4}{3},$$

$$\mathbb{E}[L_3] = 1 + \frac{1}{3} + \frac{1}{5} = \frac{23}{15},$$

$$\mathbb{E}[L_4] = 1 + \frac{1}{3} + \frac{1}{5} + \frac{1}{7} = \frac{176}{105},$$

$$\mathbb{E}[L_5] = 1 + \frac{1}{3} + \frac{1}{5} + \frac{1}{7} + \frac{1}{9} \approx 1.787302.$$

So the theorem is consistent with the earlier exact calculations.

8 A recurrence for the exact distribution

The theorem also gives a simple recurrence for the probabilities $P_n(\ell)$.

Proposition 1. *For $n \geq 2$ and $1 \leq \ell \leq n$,*

$$P_n(\ell) = \frac{2n-2}{2n-1} P_{n-1}(\ell) + \frac{1}{2n-1} P_{n-1}(\ell - 1),$$

with initial condition $P_1(1) = 1$, and with the convention that probabilities outside the valid range are zero.

Proof. From the theorem,

$$L_n = L_{n-1} + X_n,$$

where $X_n \sim \text{Bernoulli}\left(\frac{1}{2n-1}\right)$ and X_n is independent of L_{n-1}. Therefore

$$\mathbb{P}(L_n = \ell) = \mathbb{P}(L_{n-1} = \ell, X_n = 0) + \mathbb{P}(L_{n-1} = \ell - 1, X_n = 1),$$

which immediately gives the recurrence. □

8.1 Why the recurrence is useful

This recurrence is important because it turns the abstract probabilistic description into an exact computational rule. In practice, it is much easier to use this recurrence to generate the distribution than to count pairings directly.

For example,

$$P_3(1) = \frac{4}{5} P_2(1) = \frac{4}{5} \cdot \frac{2}{3} = \frac{8}{15},$$

and

$$P_3(2) = \frac{4}{5} P_2(2) + \frac{1}{5} P_2(1) = \frac{4}{5} \cdot \frac{1}{3} + \frac{1}{5} \cdot \frac{2}{3} = \frac{2}{5}.$$

So the recurrence reproduces the values found by hand in the small cases, but in a much more systematic way.

9 Further consequences

9.1 Probability generating function

Because L_n is a sum of independent Bernoulli random variables, its probability generating function is

$$G_n(t) = \mathbb{E}[t^{L_n}] = t \prod_{k=2}^{n} \left(1 + \frac{t-1}{2k-1}\right).$$

This formula is compact, but it contains the full distribution. It also makes clear once again that the distribution is built from independent Bernoulli stages.

9.2 Asymptotic growth of the mean

The expectation can be rewritten in terms of harmonic numbers:

$$\mathbb{E}[L_n] = \sum_{k=1}^{n} \frac{1}{2k-1} = H_{2n} - \frac{1}{2} H_n,$$

where $H_n = \sum_{j=1}^{n} \frac{1}{j}$.

Using the standard asymptotic formula for harmonic numbers, this gives

$$\mathbb{E}[L_n] \sim \frac{1}{2}\log n + \log 2 + \frac{\gamma}{2} \qquad \text{as } n \to \infty,$$

where γ is Euler's constant.

This confirms the earlier impression from the small cases: the mean number of loops does grow with n, but very slowly. Even when n is fairly large, the expected number of loops remains small.

9.3 Extreme probabilities

The theorem also makes it easy to compute two extreme cases.

To get exactly one loop, no loop can be formed at any stage $n, n-1, \ldots, 2$. So

$$\mathbb{P}(L_n = 1) = \prod_{k=2}^{n} \frac{2k-2}{2k-1}.$$

To get the maximum possible number of loops, namely n, a loop must be formed at every stage. So

$$\mathbb{P}(L_n = n) = \prod_{k=2}^{n} \frac{1}{2k-1}.$$

These formulas agree with the first and last entries of Table 1.

10 Python simulation and numerical checks

After proving the theoretical results, I used Python to check them numerically. I wanted the code to do more than simply produce one small example. In particular, I wanted it to test whether the exact formulas continued to agree with simulation for larger values of n.

10.1 Code representation

In the code, each original string begins as `[1,1]`. Here a 1 represents a free end and a 0 represents a part that has already been tied.

For example:

- two separate strings `[1,1]` and `[1,1]` can be joined to make `[1,0,0,1]`;
- if the two remaining free ends of a single chain are tied together, the chain is recorded as a closed loop, that is, a list containing only zeros.

I organised the code around three main functions:

1. `simulate_once(n)`, which runs one random experiment;
2. `estimate_distribution(n, trials)`, which estimates the distribution from repeated simulation;
3. `plot_distribution(n, trials)`, which compares the simulated and exact distributions graphically.

The code used in this report is available in the following GitHub repository:

https://github.com/rOOOOOnn/MA3K7-Problem-Solving-with-Python

10.2 First numerical check: $n = 5$

I first checked the exact distribution when $n = 5$. From the recurrence, the exact distribution is

$$\{1 : 0.406349,\ 2 : 0.423280,\ 3 : 0.148148,\ 4 : 0.021164,\ 5 : 0.001058\}.$$

A simulation with 10,000 trials gave

$$\{1 : 0.4066,\ 2 : 0.4212,\ 3 : 0.1473,\ 4 : 0.0232,\ 5 : 0.0017\}.$$

These values are already very close. This was a useful first check, but I did not want to stop here, because $n = 5$ is still fairly small.

10.3 Larger values of n

To test the formulas more seriously, I also simulated the process for

$$n = 10,\ 20,\ 50,\ 100.$$

For each value I compared:

- the simulated mean;
- the theoretical mean $\sum_{k=1}^{n} \frac{1}{2k-1}$;
- the asymptotic approximation $\frac{1}{2}\log n + \log 2 + \frac{\gamma}{2}$;
- the simulated variance;
- the theoretical variance $\sum_{k=2}^{n} \frac{1}{2k-1}\left(1 - \frac{1}{2k-1}\right)$.

The results are shown in Table 2.

n	Simulated mean	Theoretical mean	Asymptotic mean	Simulated variance	Theoretical variance
5	1.790200	1.787302	1.786474	0.612384	0.603437
10	2.143600	2.133256	2.133048	0.934179	0.924534
20	2.489200	2.479673	2.479621	1.282083	1.258470
50	2.952600	2.937775	2.937767	1.736953	1.709074
100	3.292000	3.284342	3.284340	2.069136	2.053142

Table 2: Comparison of simulated and theoretical moments using 10,000 trials.

What I found especially striking here is how accurate the asymptotic formula already is. Even by $n = 10$, the asymptotic mean is extremely close to the exact mean, and by $n = 50$ and $n = 100$ the agreement is almost indistinguishable at the precision shown.

This strongly supports the conclusion that the expected number of loops grows logarithmically.

10.4 Comparing the full distributions

Checking only the mean and variance is useful, but it does not prove that the entire simulated distribution matches the exact theory. To examine this more carefully, I compared the exact and simulated distributions directly using two error measures:

- the maximum absolute difference between corresponding probabilities;

- the total variation distance.

The results are shown in Table 3.

n	Max absolute error	Total variation distance
5	0.010048	0.010506
10	0.005340	0.008088
20	0.003791	0.004935
50	0.002181	0.004855

Table 3: Distance between simulated and exact distributions using 10,000 trials.

These error values are all small. So the simulation does not only reproduce the first few moments; it matches the full distribution quite well.

10.5 Plots of the distributions

I also plotted the simulated and exact distributions for $n = 10$, $n = 20$, and $n = 50$. These figures are useful because they reveal the overall shape of the distribution in a way that a table alone cannot.

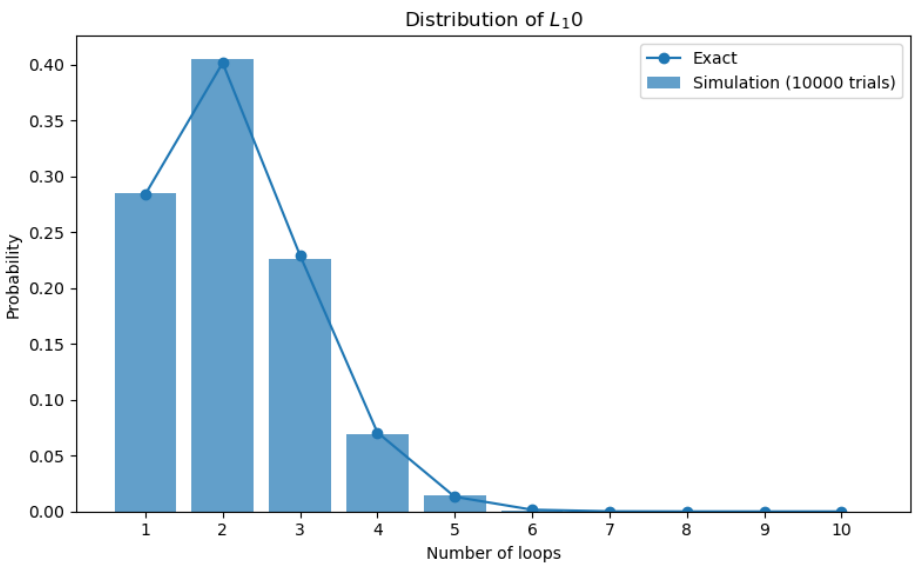

Figure 1: Simulated and exact distributions for L_{10}. Most of the probability mass is concentrated on 1, 2, and 3 loops, with the largest mass around 2 loops.

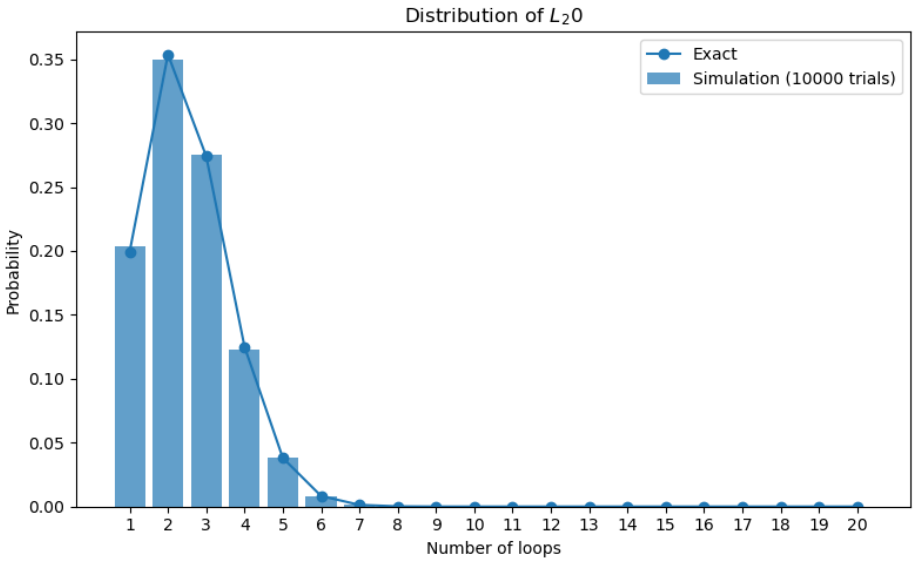

Figure 2: Simulated and exact distributions for L_{20}. The distribution has shifted slightly to the right, but it is still concentrated on small values.

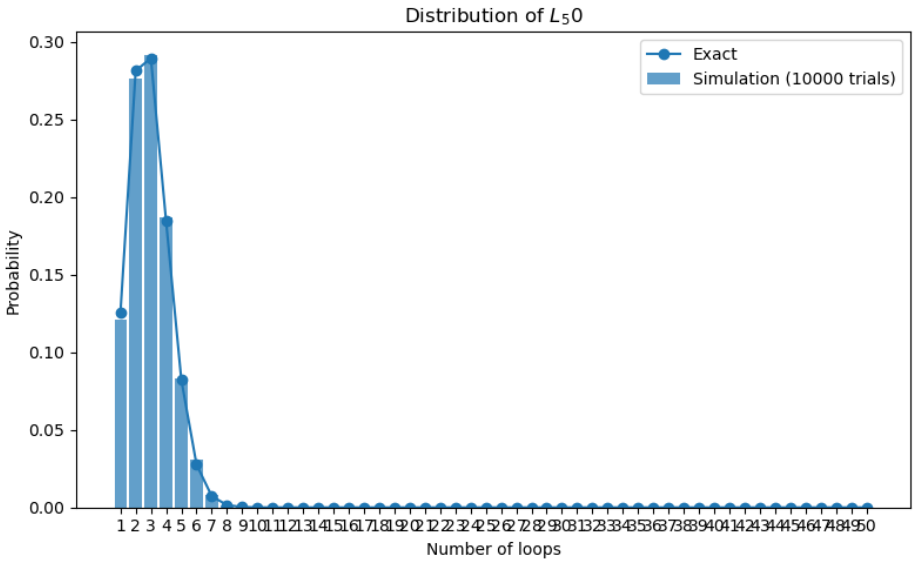

Figure 3: Simulated and exact distributions for L_{50}. Even when starting with 50 strings, the number of loops is usually still only around 2, 3, or 4, which is consistent with logarithmic growth of the mean.

These figures helped me see an aspect of the problem that is easy to miss if one only stares at formulas: although the maximum possible number of loops grows with n, the actual distribution remains heavily concentrated near the left-hand side for a long time.

11 Review: checking, reflecting, and extending

11.1 Checking the work

I checked my work in several different ways:

- by computing small exact cases directly;
- by checking that the recurrence reproduces those small cases;
- by comparing simulated and exact distributions;
- by comparing simulated and theoretical means and variances;
- by comparing the exact mean with its asymptotic approximation.

This mattered because I did not want the final result to rest on only one line of reasoning. The small-case calculations, recurrence, theorem, and Python simulations all support the same conclusion from different directions.

11.2 Reflection on the main breakthrough

The most important moment in the investigation was realising that direct counting of final pairings was not the right perspective. Initially I thought that if I pushed the counting far enough, a pattern would reveal itself. But this was not really happening. What I was getting was more data, not more understanding.

The real progress came from changing the object I was tracking. Once I started thinking in terms of open chains rather than final loops, the process became much simpler. At stage k, there are $2k$ free ends, and only one of the $2k-1$ possible partners closes the chosen chain. From that point on, the central probability $1/(2k-1)$ becomes obvious, and the rest of the theory follows naturally.

So the main lesson I take from this problem is that reformulation is often the decisive step in problem solving. In this case, the breakthrough was not a complicated combinatorial trick, but a better choice of viewpoint.

11.3 What I learned mathematically

This problem also highlighted a useful probabilistic idea: not every sum of Bernoulli-type events is binomial. Here the random variable does arise from repeated yes/no events, but the probabilities change from stage to stage, so the correct model is Poisson-binomial rather than binomial.

I found this a helpful reminder that recognising the right distributional structure matters just as much as performing calculations.

11.4 Possible extensions

There are several natural ways the problem could be extended.

- The endpoints could be chosen with a non-uniform rule rather than uniformly at random.
- Some pairings could be forbidden.
- The strings could be labelled or oriented.
- One could ask for a limiting distribution after suitable centring and scaling.

I did not pursue these here, but they suggest that the method developed in this report could be adapted to a wider class of random linking problems.

12 Conclusion

I began by specialising to small cases and computing exact probability distributions for $n = 1, \ldots, 5$. This gave useful evidence, but it also showed that direct counting was not a good general method. After getting stuck, I changed viewpoint and described the process in terms of open chains.

That reformulation led to the main result:

$$L_n \stackrel{d}{=} 1 + \sum_{k=2}^{n} X_k, \qquad X_k \sim \text{Bernoulli}\left(\frac{1}{2k-1}\right),$$

with independence. From this I obtained the exact recurrence for the distribution, formulas for the mean and variance, a probability generating function, and the asymptotic behaviour of the mean.

Finally, I used Python to test the results numerically. The simulations agreed closely with the exact theory, not only for $n = 5$ but also for much larger values such as 10, 20, 50, and 100. In particular, the numerical results supported the conclusion that the expected number of loops grows only logarithmically.

Overall, this was a good example of how specialising, conjecturing, getting stuck, reformulating, proving, and checking can all play distinct roles in solving a problem fully.